课程项目报告 / 2026年10月21日

基于封闭图形与 MediaPipe Hands的虚拟钢琴

核心技术实现与课堂演示

本项目在普通电脑浏览器中实现一套摄像头虚拟钢琴。MediaPipe提供手部关键点，程序将指尖坐标映射到屏幕上的封闭琴键区域，经悬停计时与多指占用判断后，触发钢琴采样并同步显示反馈。

当前实现C4至C5共13个琴键、双手追踪、自由弹奏与4首教学曲跟弹，支持试听、分段循环、易错片段重练和本地记录。核心交互是二维画面命中，不判断手指是否真实接触屏幕或桌面。

课堂展示要回答的问题

- 摄像头中的手指如何变成可以计算的坐标。
- 画面坐标如何与琴键对齐，并避免边界误触。
- 停留为什么只发一次声音，多根手指如何组成和弦。
- 曲库如何驱动逐音判定、目标高亮，以及独立的试听调度。

实现与验证概况

内容	当前结果
实现方式	TypeScript、Canvas 2D、Web Worker、MediaPipe、Web Audio
自动化验证	36项单元测试、19项浏览器测试覆盖通过
连续运行	原12键版本有15分钟回放记录；当前版本未重跑
人工验收	真实摄像头遮挡、真人和弦、听感与端到端延迟待实测

阅读说明：代码片段采用教学化摘录，省略部分界面提示和错误分支；函数名、关键参数和执行规则与项目实现对应。完整源码保留中文注释。

项目概览 / 操作与范围

项目功能与使用方式

演奏时，用户看着镜像画面移动双手，让指尖标记进入琴键。默认只启用两只手的食指，停留达到100毫秒后起音；离开后释放，再次进入可以重弹。

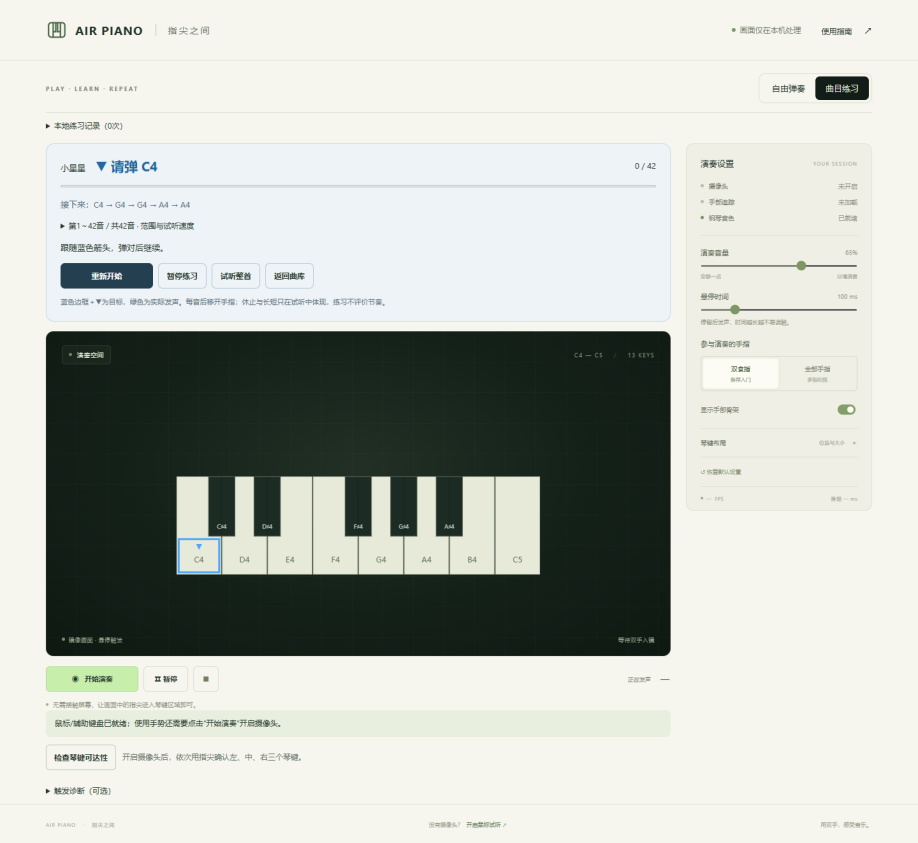


图1 当前逐音练习界面。进入练习后收起曲库，蓝色边框标记目标琴键。

操作	对应行为
开始演奏	并行准备摄像头、模型和音频，全部就绪后允许触发
暂停与继续	暂停释放音符并保留摄像头；继续后重新悬停确认
结束演奏	终止Worker，停止摄像头轨道，关闭音频上下文
鼠标与辅助键盘	无需摄像头，可练习歌曲与验证琴键音色
选曲练习	4首教学曲，弹对推进，错误停留，重复音须释放

系统设计 / 数据如何流动

系统架构与模块分工

Worker是浏览器的工作线程。系统将识别计算放在Worker中，主线程负责界面、交互和音频调度。视频与采样均从本机或当前站点处理，无需业务后端。

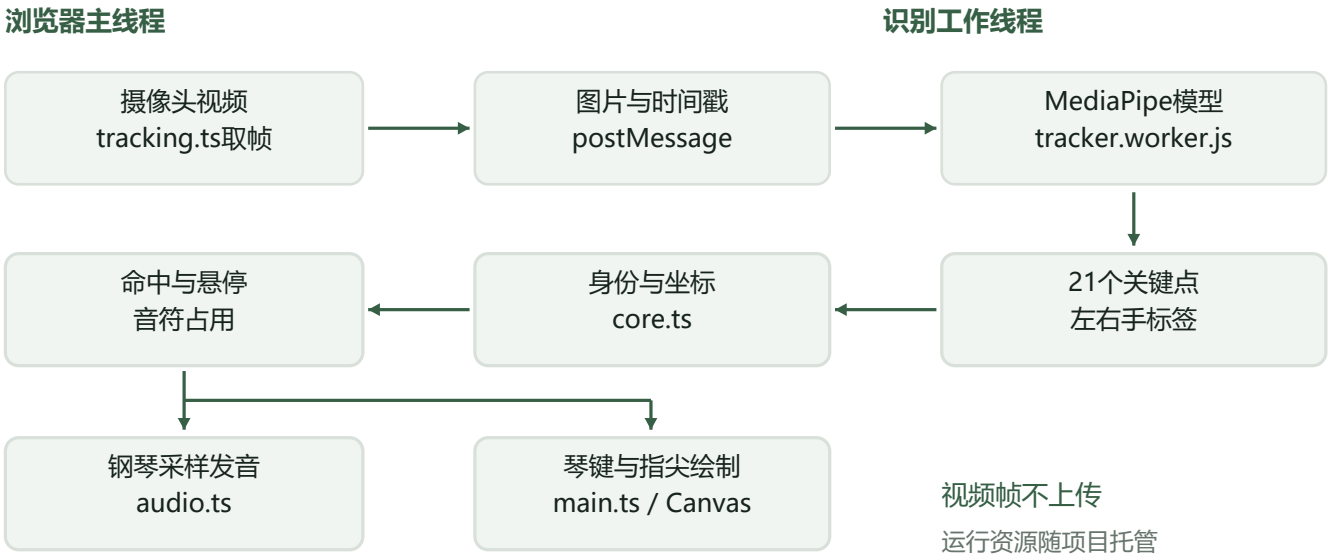


图2 从视频帧到音符事件的处理链路。Worker只提供位置，演奏规则由项目代码决定。

文件	主要职责	关键入口
main.ts	连接模块、处理按钮、绘画	start / onResult / draw
tracking.ts	摄像头与Worker桥接	openCamera / capture / stop
tracker.worker.js	模型初始化与视频推理	init / frame消息
core.ts	命中、计时、身份和占用	HoverEngine / NoteOwners
audio.ts	采样、变调、包络与清理	prepare / on / off
songs.ts / practice.ts	曲库、分段、历史数据校验	validateSong / songSection
lesson.ts	逐音状态、错误与计时	start / input / pause
song-player.ts	音频时钟试听与取消	play / stop

数据与事件分离

识别位置经规则判断后转成on/off事件，同时分发给声音和Lesson。Lesson更新进度与目标，Canvas绘制蓝框；试听走独立通道，不能推进课程。基础链路见图，新增教学分支详见第14至19页。各规则模块可独立测试。

核心技术一 / 从连续画面取得识别输入

摄像头接入与视频取帧

启动时并行准备三种资源

main.ts start函数的控制结构

```
await Promise.all([
  tracker.openCamera(),
  tracker.initModel(),
  audio.prepare(),
]);
mode = "running";
```

Promise.all同时等待三项准备操作。摄像头成功不代表模型或声音已经准备好，只有全部成功后才进入running；任意一步失败，都进入结束清理。

获得视频流并接入页面

tracking.ts openCamera函数摘录

```
const stream = await navigator.mediaDevices.getUserMedia({
  video: { width: { ideal: 640 }, height: { ideal: 480 },
           facingMode: "user" },
  audio: false,
});
this.video.srcObject = stream;
await this.video.play();
```

getUserMedia返回持续的视频流。ideal表示期望分辨率，不保证设备一定采用该值；audio:false表示不申请麦克风。视频通过srcObject交给video元素播放，并在localhost或HTTPS环境下申请权限。

只提交可以及时处理的新帧

capture检查四类条件：上一个任务仍在进行、视频未就绪、当前帧时间与上次相同、距离上次取帧不足30毫秒。任何条件成立就跳过。createImageBitmap将当前画面转换成图片，宽度缩放为640，高度保持原比例。

pending标志贯穿取图与推理阶段，使同一时间最多一个任务在途。图像通过postMessage及转移列表交给Worker；识别完成后恢复空闲，再处理后续画面。它优先保持结果新鲜，不通过排队补算旧画面。

核心技术二 / 识别任务独立运行

MediaPipe推理与线程通信

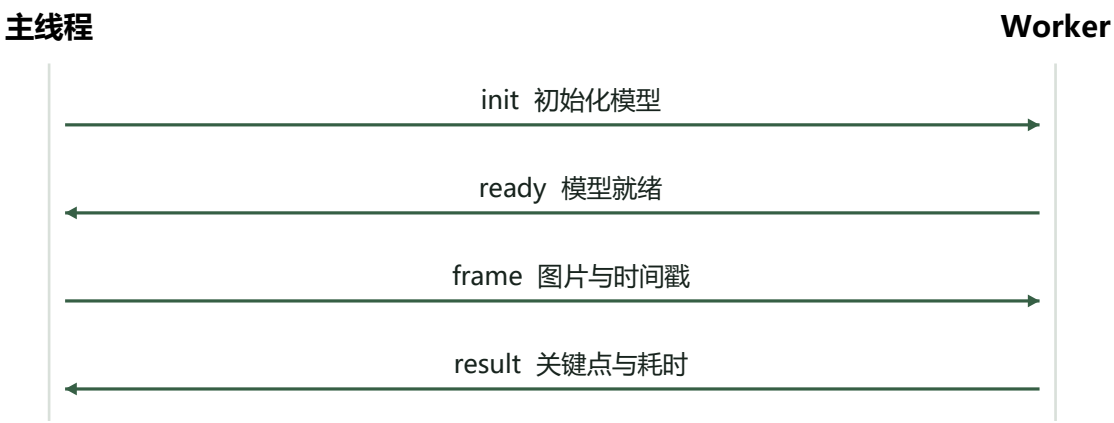


图3 主线程与Worker的消息协议。错误统一以error消息返回。

tracker.worker.js 配置结构摘录，modelAssetPath由本地URL生成

```
const options = delegate => ({
  baseOptions: { modelAssetPath, delegate },
  canvas: new OffscreenCanvas(1, 1),
  runningMode: "VIDEO",
  numHands: 2,
  minHandDetectionConfidence: 0.55,
  minHandPresenceConfidence: 0.55,
  minTrackingConfidence: 0.55,
});
```

模型采用预训练Hand Landmarker，最多输出两只手。代码优先尝试GPU，初始化失败后回退CPU。三项阈值分别参与检测、手部存在和跟踪判断，不能解释成识别准确率55%。[1]

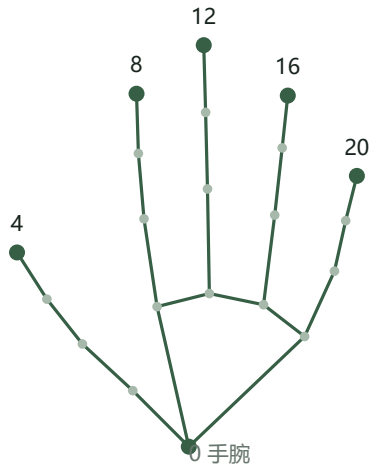
每帧推理与结果清理

detectForVideo(bitmap, timestamp)同步执行识别，因此放在Worker中，减少对按钮与绘画的阻塞。结果消息包含关键点、左右手标签、原始取帧时间戳和本次推理耗时。主线程忽略超过200毫秒的旧结果。

WASM用于在浏览器中执行模型计算，OffscreenCanvas是离屏画布。Worker使用经典脚本以支持当前WASM加载器的importScripts。图片在finally中调用close，无论成功还是失败都释放；转移列表避免主线程额外复制整张图片。[2]

核心技术三 / 多帧数据保持连续

指尖提取与稳定身份关联



4 拇指指尖

8 食指指尖

12 中指指尖

16 无名指指尖

20 小指指尖

示意图用于说明编号，不表示实际识别结果。

图4 手部关键点编号示意。每只手有21个关键点，交互主要使用五个指尖。[1]

main.ts 指尖选择与输入编号结构

```
const tips = settings.fingers === "all"
  ? [4, 8, 12, 16, 20] : [8];
const fingerId = `hand:${hand.id}:${tip}`;
```

默认取每只手的8号点，即双食指模式；全部手指模式取五个指尖。hand.id由HandTracker分配，tip为关键点编号，二者组合后每根手指独立计时。

为什么不能直接使用数组下标

前后两帧的检测数组可能从“左手、右手”变成“右手、左手”。HandTracker比较新旧手腕的欧氏距离，并在左右手标签不一致时增加0.04的比较代价。最近候选代价小于0.23且没有明显歧义时复用身份；两个最近候选差值小于0.035时创建新身份。以上距离均基于模型归一化坐标。

平滑坐标而不预测遮挡位置

core.ts 平滑公式的等价教学写法，dt为距上次观察的毫秒数

```
const alpha = 1 - Math.exp(-Math.max(1, dt) / 35);
const x = old.x + (current.x - old.x) * alpha;
```

35毫秒是平滑时间常数，间隔越长越信任新位置。若旧值0.40、新值0.44、alpha为0.5，平滑值为0.42。短时轨迹达到350毫秒未更新会移除，关联不明确时不强行沿用旧状态。

核心技术四 / 把相对位置换成画布位置

坐标映射与镜像对齐

模型图像坐标以宽高归一化，画布逻辑坐标使用CSS像素。mapPoint必须使用与视频object-fit:contain相同的缩放和留白，并处理镜像。当前交互仅使用x和y，不把相对深度z当作真实接触距离。[1]



图5 640×480视频放入1000×1000显示区域后的坐标计算示例。

core.ts mapPoint函数的等价展开，变量名对应计算步骤

```
const scale = Math.min(width / sourceWidth,
                       height / sourceHeight);
const offsetX = (width - sourceWidth * scale) / 2;
const offsetY = (height - sourceHeight * scale) / 2;
const mappedX = offsetX + (mirror ? 1 - p.x : p.x)
  * sourceWidth * scale;
const mappedY = offsetY + p.y * sourceHeight * scale;
```

示例中缩放比例为1.5625，有效视频为1000×750，上下各留125像素。点(0.25,0.50)镜像后横向比例为0.75，最终为(750,500)。如果漏算留白，标记会整体偏移；如果视频镜像但坐标未反转，左右会错位。

keyboardGeometry先求实际视频内容矩形，再在其中生成琴键。画布背板按设备像素比DPR提高分辨率，但逻辑坐标仍用CSS像素，避免重复放大指尖或命中区域。

核心技术五 / 绘制与交互使用同一几何

封闭琴键与图形命中检测

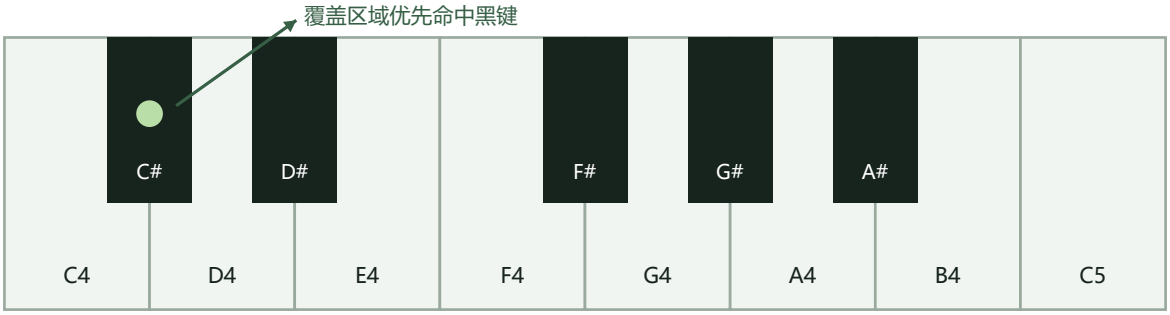


图6 黑键覆盖白键的一部分，重叠区域必须优先命中黑键。

PianoKey保存x、y、w、h、note、midi和black。绘图与命中判断都读取这些字段，避免可见琴键与交互区域不一致。键盘从MIDI音高60到72，共8白键与5黑键，白键偏移为[0,2,4,5,7,9,11,12]，黑键偏移为[1,3,6,8,10]。

core.ts makeKeys布局计算

```
const w = width * layout.width / 100;
const h = height * layout.height / 100;
const x = (width - w) * layout.x / 100;
const y = (height - h) * layout.y / 100;
const step = w / 8;
```

默认宽度84%、高度30%；x=50表示剩余横向空间居中，y=64表示剩余竖向空间中的64%位置。白键左右各留1像素间隔，黑键宽度为step的58%，高度为白键的61%。

core.ts contains函数的四条边界判断

```
return p.x >= r.x - margin &&
       p.x <= r.x + r.w + margin &&
       p.y >= r.y - margin &&
       p.y <= r.y + r.h + margin;
```

满足四个不等式就表示点位于矩形内。margin为正时扩大区域，为负时收缩。琴键数组先白后黑，hitKey复制数组后反向查找，使黑键优先，同时不改变绘图顺序。

核心技术六 / 解决进入琴键却不响的问题

悬停确认与自适应容错

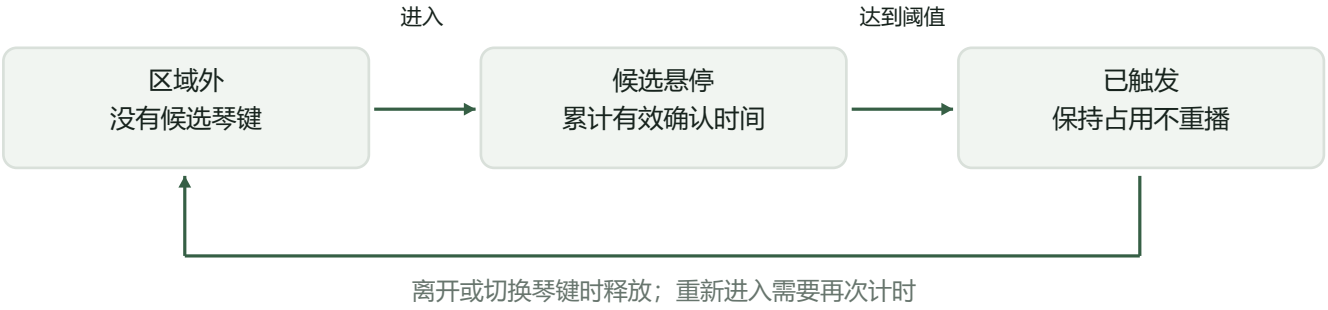


图7 区域外、候选、已触发三态。短暂漏检冻结进度，起音只能发生在有效识别帧中。

core.ts HoverEngine.update中的关键摘录

```
if (!state.interrupted) {
  state.confirmedMs += Math.min(120,
    Math.max(0, now - state.lastSeen));
}
if (target && !state.active &&
    state.confirmedMs >= this.dwell) {
  state.active = true;
  this.notes.set(finger.id, target.midi);
}
```

confirmedMs表示被有效观察支持的停留时间；active表示本次进入是否已发声。默认阈值100毫秒。每次观察最多增加120毫秒，避免一次迟到结果把很长的空白时间全算进来。圆环读取confirmedMs，不随绘画时钟自动填满。

漏检示例：进度冻结，不借用空白时间

0 ms进入C4，进度0；70 ms再次观察到，累计70；100 ms漏检时冻结；140 ms返回第一帧仍为70；175 ms继续有效观察，累计105毫秒，才发声一次。即使绘画循环不断运行，也不能在手指消失时自动起音。

容错有上限，缺少画面时不能自动发声

识别间隔使用指数均值：cadence = 0.65 × old + 0.35 × delta。候选容忍窗口为clamp(cadence × 2.5, 160, 320)毫秒；已起音的释放等待为max(200, graceMs)。超时、明确离开或切键后重新确认。有效结果间隔150毫秒时仍能确认；超过200毫秒帧龄的旧结果仍被主线程丢弃。

进入边界收缩最多2像素，保持区域放宽键宽的3.5%。黑键边缘未满足进入条件时不回落到其下方白键。以上措施改善稳定性，但不保证弱光、遮挡下没有漏触。

核心技术七 / 避免同音互相打断

多指占用与和弦管理

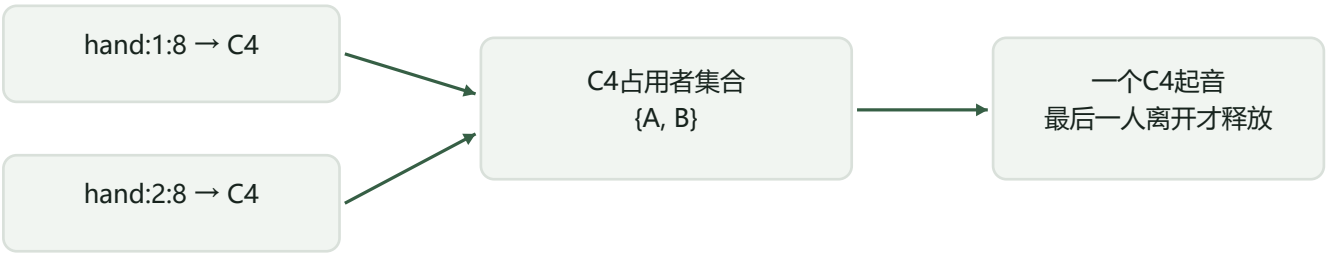


图8 两根手指占用同一音符时，起音一次；最后一个占用者离开才释放。

NoteOwners使用Map保存“输入编号→音高”。同一音高可以对应多个输入，但声音只在首次占用时起音。手指、鼠标、键盘都使用同一个管理器，因而能够共享释放规则。

core.ts NoteOwners.set摘录，前面还会跳过“同输入同音不变”的情况

```
const old = this.owners.get(owner);
this.owners.delete(owner);
if (old !== undefined &&
    ![...this.owners.values()].includes(old))
  this.emit({ type: "off", midi: old });
if (midi !== undefined) {
  const occupied = [...this.owners.values()].includes(midi);
  this.owners.set(owner, midi);
  if (!occupied) this.emit({ type: "on", midi });
}
```

set(owner, midi)代表按下或切换，set(owner)省略音高代表释放。先解除旧占用，再判断新音是否首次被占用；emit将on/off事件交给main.ts，再调用audio.on或audio.off。

占用变化	事件	结果
无人→A占用C4	on(60)	开始C4
A→A和B共同占用C4	无	不重复起音
A和B→只剩B	无	C4占用仍保留
只剩B→无人	off(60)	释放C4

不同音符则各有独立播放实例，例如C4、E4、G4、B4可同时工作。reset先清空占用，再逐音发送off，使界面回调读取到的活动列表也是空的。

核心技术八 / 将音符事件变成声音

钢琴采样播放与音量包络



图9 Web Audio声音节点通路。单音包络与总音量分别控制。

prepare通过fetch读取五个钢琴MP3采样，再用decodeAudioData解码成AudioBuffer。五个采样全部成功后才替换正式缓冲表，避免半加载状态进入演奏。音频上下文在用户点击时resume，以启用浏览器声音。

audio.ts on函数的教学化摘录，省略实例登记与结束回调

```
const sample = [...this.buffers.keys()].sort(
  (a, b) => Math.abs(a - midi) - Math.abs(b - midi)
)[0];
const source = context.createBufferSource();
source.buffer = this.buffers.get(sample);
source.playbackRate.value = 2 ** ((midi - sample) / 12);
const gain = context.createGain();
gain.gain.setValueAtTime(0, context.currentTime);
gain.gain.linearRampToValueAtTime(
  0.24, context.currentTime + 0.008);
source.connect(gain);
gain.connect(this.master);
source.start();
```

采样为C4、D#4、F#4、A4、C5，其余音符选择最近采样并变调。本页#表示升号。目标A#4编号70，最近A4编号69，速率约为2的1/12次方，即1.059；高一个八度时速率为2。[3]

起音与释放都采用渐变

单音在8毫秒内从0渐入到0.24，减少突变噪声；释放时cancelAndHoldAtTime保留当前音量，再用120毫秒淡出，130毫秒后停止播放源。每次起音创建新的BufferSource，结束回调只清理自身，避免旧尾音删除后来重弹的新实例。

持续停留只保持音符占用，采样会自然衰减，不循环重播。素材为Alexander Holm的Salamander Grand Piano，经Tone.js Audio分发，适用CC BY 3.0，应用与资源目录保留署名。[4]

工程实现 / 反馈与交互保持一致

界面绘制与设置保存



同一个stage容器定位

同一套缩放与留白规则

DPR只改变背板像素

绘画循环不等于推理频率

图10 视频自行播放，Canvas负责覆盖图形，两层使用同一坐标变换。

main.ts draw函数的简化结构

```
function draw(now: number) {
  ctx.clearRect(0, 0, width, height);
  if (mode === "running") {
    tracker.capture(now);
    hover.expire(now);
  }
  // draw keys, skeleton and fingertip progress
  requestAnimationFrame(draw);
}
```

requestAnimationFrame让浏览器安排下一次绘画；capture内部再次检查限速与忙碌状态。识别变慢时画面仍可刷新。active决定发音高亮，candidate表示候选悬停；进度环从0画到1，反馈确认过程。

逻辑像素与高分屏背板分开

resize读取舞台的CSS宽高，画布背板乘以DPR，并设置对应变换；DPR最多取2。琴键和指尖仍使用CSS像素，因此不需要再乘一次DPR。窗口或布局变化后重算几何并清理旧交互。

设置由内存同步到本地存储

settings保存音量、悬停时间、手指模式、骨架开关与布局。控件事件先修改实际参数，再同步文字并用JSON.stringify写入localStorage。readSettings校验类型和范围，损坏JSON回退默认值。修改布局、手指模式或悬停时间会重置交互；音量与骨架开关无需重新生成琴键。

工程实现 / 摄像头权限与资源生命周期

暂停、继续与真正关闭摄像头

本项目的暂停是暂停识别、发声和课程，不是关闭摄像头。暂停保留MediaStream和Worker，摄像头指示灯可能仍亮；继续复用现有连接。只有“结束演奏”才停止视频轨道并关闭识别线程与音频。

操作	摄像头与课程	授权行为
首次开始	打开视频流，并行准备模型与音频	调用getUserMedia，浏览器处理权限
任一暂停入口	释放音符，停止提交识别帧；课程保留进度	不调用getUserMedia，轨道仍live
任一继续入口	复用视频流和Worker，重新确认指尖	不重新申请摄像头
结束	轨道ended、Worker终止、音频关闭	释放设备连接
结束后重新开始	建立新会话与新视频流	重新调用接口，是否弹窗取决于权限设置

main.ts 两个暂停入口共同调用togglePause

```
function togglePause() {
  if (lesson.state === "paused" && practicing)
    void prepareLearning("resume");
  else if (mode === "running") pause();
  else if (mode === "paused") void resume();
  else if (lesson.state === "paused")
    void prepareLearning("resume");
  else if (lesson.active) suspendLearning();
}
```

优先检查课程暂停，是因为可达性检查可能只暂停课程而保留识别运行。若先检查摄像头running，用户点击“继续”可能反而进入摄像头暂停。恢复课程只准备音频、清理旧输入并恢复状态，不调用openCamera。

旧任务失效与结束清理

token、generation及lessonGeneration隔离迟到的授权、音色加载和切歌任务。结束时先更新代次，再停止轨道、断开video.srcObject、终止Worker和关闭AudioContext；迟到授权返回的流也要停止。后台或失焦自动暂停，返回后由用户主动继续。模型初始化超时45秒，在途推理超时5秒后清理并提示。

验证：交替使用两个暂停入口两轮后，getUserMedia仍只调用1次，流对象身份相同、轨道保持live；点击结束后变为ended。测试使用合成摄像头，不能据此保证所有浏览器在重新启动时都不弹权限框。

教学功能一 / 用数据驱动歌曲

曲库模型与曲谱校验

歌曲不是整段MP3背景音，而是一组可计算的音符步骤。songs.ts定义Song与SongStep：MIDI编号表示音高，beats表示以四分音符为单位的拍数，midi为null表示休止。

songs.ts 实际接口与教学示例；60=C4，67=G4

```
interface SongStep {
  midi: number | null;
  beats: number;
}
// A teaching example, not the full song:
const steps = [
  { midi: 60, beats: 1 },
  { midi: 60, beats: 1 },
  { midi: 67, beats: 2 },
  { midi: null, beats: 1 },
];
```

内置曲目	实际音符数 / 音域	教学重点
小星星	42 / C4-A4	同音重复与移开重弹
欢乐颂	30 / C4-G4	C大调主题节选，相邻键移动
两只老虎	32 / C4-C5	教学改编，末句低音G移高八度
指尖小阶梯	26 / C4-C5	项目原创，含黑键与中途休止

为什么要校验音域

原版仅有C4至B4的12键，C5采样存在不代表界面有C5。当前新增第8个白键后覆盖MIDI 60至72。validateSong拒绝超出范围、非整数音高、无音符、无效时值和速度，不把超范围旋律静默折回八度。

曲谱须有1至1000步、至少一个非休止音，单步拍数大于0且不超过16，速度30至240 BPM。练习使用practiceNotes过滤休止，试听保留时值。曲名、难度、音域和来源在界面展示，明确是教学简化版或改编版，不声称等同原谱。

教学功能二 / Lesson纯逻辑状态机

逐音判定与连续同音防连过

识别模块确认用户输入后，由NoteOwners发出有效on/off事件。main.ts同时把事件交给PianoAudio与Lesson.input。Lesson不读摄像头，也不根据琴键颜色判断成绩，所以鼠标、辅助键盘和手势可使用同一套规则。

状态	含义与变化
idle	未开始；start提取当前范围音符并重置统计
playing	等待目标on；错误保留进度，正确消费一步
release	正确后等待held集合清空；其他起音不推进
paused	保留进度与已用时间，清空占用，等待主动恢复
completed	最后一个音正确后冻结结果，等待重练或选曲

lesson.ts 起音分支摘录，省略短暂错误提示字段

```
// Excerpt: duplicate presses and release gate.
if (this.held.has(event.midi)) return;
this.held.add(event.midi);
if (this.state === "release") return;
if (event.midi !== this.target) {
  this.mistakes++;
  this.mistakeSteps.add(this.index);
  this.attempted = true;
  return;
}
```

以C4、C4、G4三个音为例

第一次C4起音完成第1步；持续停留不产生新on，也不能完成第2步。释放后held为空，状态回到playing；再次进入C4才完成第2步。目标G4时提前按住其他键，不能靠持续占用自动完成。错误按键仍正常发声，只是不推进进度。

成绩不是最终弹对数量除以总数

等待模式最终都会弹对，因此首次命中率定义为firstTry / notes.length。第一次尝试就正确才增加firstTry；本步骤先弹错再弹对不增加。有效用时用performance.now累积，排除暂停，完成后冻结。不评价节奏、休止、长音或和弦。

教学功能三 / 用音频时钟调度声音

整首试听与练习输入隔离

试听与练习互斥。自动发声走PianoAudio.schedule的独立播放源集合，不经过NoteOwners或Lesson.input；否则程序会把自己播放的音当成用户输入，自动完成课程。试听中阻止用户演奏输入，可随时停止。

song-player.ts 调度每一步的主要计算，实际代码位于map内

```
let cursor = this.audio.time + 0.08;
const end = cursor + step.beats * 60 / song.bpm / speed;
if (step.midi !== null) {
  this.audio.schedule(step.midi, cursor,
    end - Math.min(0.06, (end - cursor) * 0.1));
}
cursor = end;
```

机制	实现目的
音频时钟	AudioContext.currentTime确定声音的实际起止
提前80 ms预排	短曲一次安排完；定时器抖动不直接改变音符起止
25 ms界面定时器	读取当前时钟，更新当前音符或休止显示
独立scheduled集合	取消试听不会混淆用户voices中的按住状态
播放停止或切歌	停止已排入时钟的播放源，清除定时器与目标
中途调度异常	catch中也调用stop，避免部分音符继续响

速度改变时值，不改变音高

例如120 BPM下的一拍是0.5秒，0.5倍试听变为1秒。speed只作用于时间计算，不修改音高或采样变调比例。界面提供0.5、0.75、1、1.25倍；改变速度时停止旧试听，重新点击后从所选范围开头播放。

休止同样推进cursor但不安排播放源。音符尾部保留短间隔区分重复音，释放采用120毫秒淡出；25毫秒定时器仅用于显示，不是精确节拍发生器。

教学功能四 / 从完整曲谱到局部练习

分段循环与易错片段重练

界面使用从1开始的音符编号，内部使用从0开始、含头不含尾的[start,end)范围。编号只统计非休止音符；改变范围先校验，再重置当前进度。练习与试听共用currentPracticeSong，避免练的是一段、听的是另一段。

practice.ts songSection截取步骤的核心代码

```
let index = -1;
const steps = song.steps.filter(step => {
  if (step.midi !== null) index++;
  return index >= start && index < end &&
    !(index === end - 1 && step.midi === null);
});
```

例如原步骤为C4、休止、D4、E4，选第1至2音得到C4、休止、D4。保留段内休止和拍数，排除段前与最后一音后的休止；起始大于结束、越界或小数编号均拒绝。

循环必须等最后一音释放

完成最后一个on后先保存结果，不能立即起下一轮。等NoteOwners.active为空且勾选循环、allowLoop为true时才调用Lesson.start。每轮独立记录；suspendLearning先将allowLoop置false，clearingNotes标记程序清理，防止暂停或切歌产生的off事件意外重启。

边界情况	处理规则
最后一音仍按住	显示完成，等待全部释放后才循环
暂停/切歌/结束	先取消循环意图，再释放声音
没有任何错音	不显示易错片段重练按钮
一个或多个步骤出错	mistakeSteps使用Set，重复错误不重复记录步骤
点击易错片段	取首次至末次错误范围，尽可能加前后各一音上下文

这不是自动识别乐句结构，而是基于错误位置的局部范围练习。若错误分散，重练片段可能较长；后续可增加人工乐句标记，但当前实现不宣称已具备乐句分析。

教学功能五 / 成绩怎样保存与比较

练习记录与本地数据校验

完成时保存歌曲编号、完成时间、起止范围、错误次数、首次正确数和有效用时。数据写入localStorage的air-piano:history:v1，最多保留最近50轮，列表展示最近10轮。不需要账号、服务器或上传。

main.ts saveCompletion主要结构；实际包含写入失败提示

```
if (lesson.state !== "completed" || recorded) return;
recorded = true;
history = [{
  songId: selectedSong.id, at: Date.now(),
  start: sectionStart, end: sectionEnd,
  mistakes: lesson.mistakes, firstTry: lesson.firstTry,
  elapsedMs: lesson.elapsedMs,
}, ...history].slice(0, 50);
localStorage.setItem(HISTORY_KEY, JSON.stringify(history));
```

为什么还需要recorded

同一轮完成后还有最后一次off和界面重画。recorded确保只保存一次；开始新曲或下一轮时复位。循环的每一轮也单独保存，不能把一轮的off记成新成绩。

检查	目的
JSON解析与数组类型	损坏数据回退空记录，不阻止演奏
已知歌曲与合法范围	防止旧数据或篡改数据引用不存在的曲目
次数、时长、日期边界	过滤负数、非整数与无效日期
最多50条与字段白名单	约束数据规模，丢弃额外字段
textContent渲染	存储中的文本不能作为HTML执行
相同曲目和范围比较	避免2音短段与42音整曲的命中率混比

记录可一键清除，不影响音量、布局等设置。存储不可用时仍允许本次练习并提示；刷新后不会恢复尚未完成的进度。本地记录也不等于不可篡改的考试成绩。

交互反馈 / 解释系统正在等待什么

目标高亮、诊断与可达性引导

画面信号	来源与含义
蓝框与向下箭头	Lesson.target或试听当前音；不是实际按下
绿色琴键	NoteOwners.active，用户输入已确认
指尖圆环	confirmedMs / dwell，缺少有效观察不会自行增长
橙色错误边框	错误音短暂提示500毫秒，不推进课程
后续五音预览	独立提示条，不同时高亮多个未来键

输入提示按状态给出原因

main.ts每150毫秒刷新简明提示，按准备中、暂停、试听、音色/静音、摄像头状态、延迟、无手、候选、已触发、未命中依次判断。已经触发但声音自然衰减时，明确提示移开后重弹；仅有手部轮廓不代表食指中心已经命中。

诊断信息默认收起

展开后才显示识别间隔、漏检容忍窗口、推理耗时、帧龄、旧结果跳过次数、起音与超时计数、指尖当前音及确认毫秒数。原因字符串帮助区分切键、离开和超时；不记录画面，不保存诊断历史。

先检查左、中、右是否能到达

开启摄像头后点击“检查琴键可达性”，依次用指尖确认C4、G4、C5。检查会保留课程暂停，单独允许手势；鼠标和辅助键盘被阻止，不能替代通过。够不到时用已有布局滑块调整位置和大小；检查可随时停止，再继续原课程。

开始练习后集中显示当前任务

曲库支持名称、难度和音域筛选。开始练习或试听后收起曲库与顶部介绍，缩短舞台高度，让当前目标和琴键集中显示；进阶选项与记录使用可展开区域。返回曲库会清空当前进度，历史完成记录保留。

可达性检查是用户引导，不是自动相机标定；它不修改模型坐标、不推算真实桌面深度，也不自动扩大琴键。

验证结果 / 区分功能证据与设备体验

自动化测试与性能采样

验证项目	记录结果
单元测试	36项：含容错、重复音、分段、曲谱、计时和本地数据
浏览器测试	19项覆盖：真实模型、整曲、循环、切歌、权限复用等
资源完整性	15个运行资源检查SHA256与MediaPipe版本
生产构建	TypeScript与Vite构建通过，运行资源随静态站点分发
历史连续回放	2026-10-20原12键版，15分钟30次采样；当前版未重跑

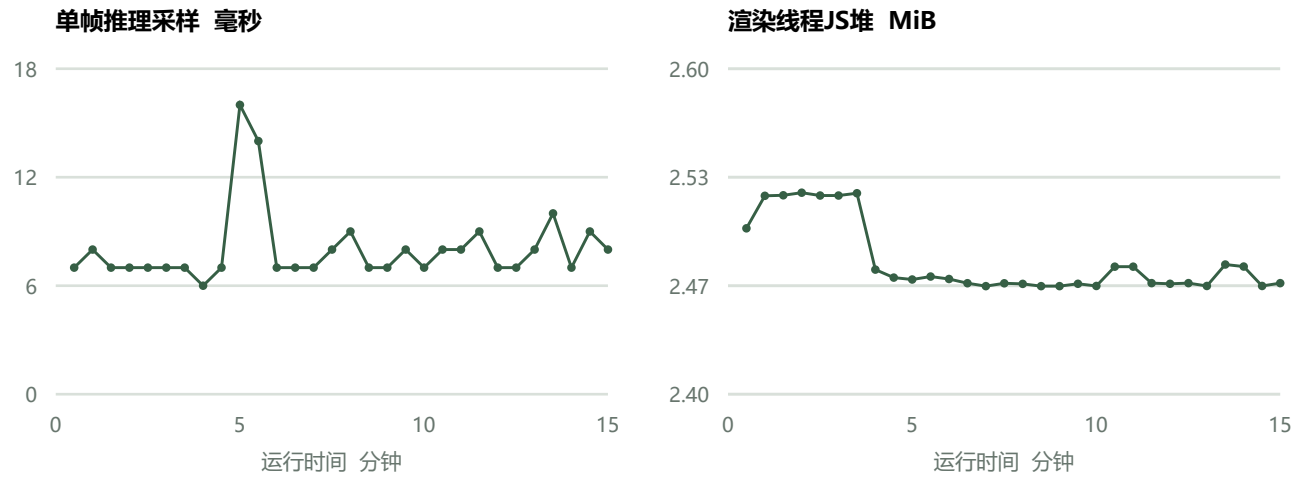


图12 原12键版本的历史连续回放记录，约每30秒采样。不是当前版本重测结果。

历史性能采样环境为Windows 10.0.26200、Intel Core Ultra 9 275HX、Edge 155.0.4283.62，无头浏览器。输入采用官方手部图片形成的视频流，模型与Web Audio接口真实运行。

采样识别更新率30～31 FPS，稳定阶段单帧推理采样6～16毫秒；渲染线程JS堆经垃圾回收后约2.47～2.52 MiB，采样未见持续上升。内存数值按1024^2字节换算，不覆盖Worker和GPU全部内存。

结果边界：这些数字不等于真人弹奏准确率或动作到出声延迟。浏览器测试验证了采样解码、非零信号和播放入口；实际扬声器听感、真人四指遮挡和实体摄像头表现仍待人工实测。推理耗时不包含悬停等待、取帧及音频输出。

展示建议 / 约12分钟，按功能闭环演示

课堂演示顺序与答辩要点

时长	演示操作	核心讲解
1分钟	自由弹奏C4，移开重弹	摄像头二维命中，持续停留只起音一次
2分钟	架构、指尖与坐标图	模型输出关键点；项目代码负责映射与规则
2分钟	停留、漏检与诊断	有效确认时间、容错上限，不能丢手自动发声
2分钟	小星星前两个C4	弹对后等待释放，连续同音必须重新触发
2分钟	短段试听与循环	试听通道独立，变速只改时间，不自动完成
1分钟	完成与查看本地记录	首次命中率、有效用时、相同范围比较
1分钟	暂停、继续、结束	暂停保留连接；结束才关闭摄像头
1分钟	说明证据与局限	36项单元、19项浏览器覆盖；真人表现待测

演示前准备与备选方案

执行npm run dev，用桌面Chrome或Edge打开本地地址。检查系统音量、摄像头权限、光照与采样状态；先使用双食指，完成C4、G4、C5可达性检查。现场识别不稳时，用鼠标/辅助键盘演示曲库判定，并明确它不能代替手势稳定性验收。

建议预先设置一个两音短段

小星星选择第1至2音，两者都是C4。先弹错D4展示进度不变，再两次弹C4并分别释放；完成结果应是首次命中1/2，即50%。勾选循环后再次完成，释放最后一个键才进入下一轮。

答辩时避免混淆的三个概念

是否训练了自己的模型。没有，使用MediaPipe预训练模型；项目完成交互、教学、音频与资源调度。

蓝色高亮是否代表已经按下。不代表。蓝色是目标，绿色是实际确认输入，圆环是确认进度。

暂停是否关闭摄像头。当前设计不关闭。暂停停止识别和发声，保留连接；结束才停止轨道。

交付边界 / 后续工作

局限 改进与参考资料

当前局限与改进顺序

普通RGB摄像头对遮挡、弱光和快速运动较敏感；双手交叉可能造成身份关联不确定。当前通过重新编号、计时确认和超时释放降低异常影响，但不能保证所有姿态都稳定。

建议先在目标电脑完成逐键命中、同键20次重弹、真人两指及四指和弦和15分钟真实演奏验收，并用外部录音录像测量动作到出声。随后依据记录调整平滑、悬停和边界参数，避免只凭主观感觉修改。

已完成曲谱教学与分段练习；后续可增加更多八度、用户绘制多边形琴键、节奏评价或导入MIDI，但不属于当前实现。下压识别应采用独立策略并增加姿态校准；力度与真实桌面接触检测需另外验证，不能将当前二维规则直接改名为真实触摸。

源码阅读与定位

关注的问题	源码入口
位置怎样对齐	core.ts → mapPoint；main.ts → keyboardGeometry
为什么只起音一次	core.ts → HoverEngine.update / expire
同音多指怎样释放	core.ts → NoteOwners.set / reset
声音怎样产生	audio.ts → prepare / on / off / close
课程怎样推进	lesson.ts → input；main.ts → renderLesson
片段与试听怎样生成	practice.ts → songSection；song-player.ts → play
资源怎样关闭	main.ts → end；tracking.ts → stop

参考资料与项目证据

[1] Google AI Edge Hand Landmarker Web文档

[2] MDN Worker postMessage与转移列表

[3] MDN AudioBufferSourceNode playbackRate

[4] Tone.js Audio Salamander采样及许可声明

项目证据：docs/验收记录.md、docs/开发说明.md、tests/core.test.ts、tests/lesson.test.ts、tests/practice.test.ts与tests/browser/。功能测试对应2026年10月21日记录；性能图来自2026年10月20日原12键版历史数据。

课堂展示结束后可按“命中正确、起音一次、移出释放、重新进入可重弹、结束后设备关闭”的顺序邀请现场验证。